So *attack vectors for Header Injection are based on the injection of CRLF characters in a header field.* And what could an attacker do with a false redirection? They could redirect to a phishing site that looks the same as yours, but ask to login again (and sends the login credentials to the attacker). Or they could install malicious software through browser security holes on that site. Rails 2.1.2 escapes these characters for the Location field in the `redirect_to` method. *Make sure you do it yourself when you build other header fields with user input.*

7.8.1 Response Splitting

If Header Injection was possible, Response Splitting might be, too. In HTTP, the header block is followed by two CRLFs and the actual data (usually HTML). The idea of Response Splitting is to inject two CRLFs into a header field, followed by another response with malicious HTML. The response will be:

```
HTTP/1.1 302 Found [First standard 302 response]
Date: Tue, 12 Apr 2005 22:09:07 GMT
Location: Content-Type: text/html


HTTP/1.1 200 OK [Second New response created by attacker begins]
Content-Type: text/html


&lt;html&gt;&lt;font color=red&gt;hey&lt;/font&gt;&lt;/html&gt;
[Arbitary malicious input is
Keep-Alive: timeout=15, max=100         shown as the redirected page]
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html
```

Under certain circumstances this would present the malicious HTML to the victim. However, this only seems to work with Keep-Alive connections (and many browsers are using one-time connections). But you can't rely on this. *In any case this is a serious bug, and you should update your Rails to version 2.0.5 or 2.1.2 to eliminate Header Injection (and thus response splitting) risks.*

8 Unsafe Query Generation

Due to the way Active Record interprets parameters in combination with the way that Rack parses query parameters it was possible to issue unexpected database queries with `IS NULL` where clauses. As a response to that security issue (CVE-2012-2660, CVE-2012-2694 and CVE-2013-0155) `deep_munge` method was introduced as a solution to keep Rails secure by default.

Example of vulnerable code that could be used by attacker, if `deep_munge` wasn't performed is:

```
unless params[:token].nil?
  user = User.find_by_token(params[:token])
  user.reset_password!
end
```

When `params[:token]` is one of: `[nil]`, `[nil, nil, ...]` or `['foo', nil]` it will bypass the test for `nil`, but `IS NULL` or `IN ('foo', NULL)` where clauses still will be added to the SQL query.

To keep rails secure by default, `deep_munge` replaces some of the values with `nil`. Below table shows what the parameters look like based on `JSON` sent in request:

JSON	Parameters
`{ "person": null }`	`{ :person => nil }`
`{ "person": [] }`	`{ :person => [] }`